



S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 02

Aim: To understand and demonstrate the use of basic commands in different operating systems (Windows, Linux, and UNIX) for managing files, directories, permissions, and user interactions through a terminal or command-line interface.

Name: Parth

Bhatpalliwar

USN: CM24029

Semester / Year: IV Sem / II Year

Academic Session: 2025-26

Date of Performance: 20-01-26

Date of Submission:

- ❖ **Aim:** To understand and demonstrate the use of basic commands in different operating systems (Windows, Linux, and UNIX) for managing files, directories, permissions, and user interactions through a terminal or command-line interface.

❖ **Objectives:**

1. To learn and practice fundamental command-line operations for file and directory management.
2. To explore and utilize user and permission management commands effectively.
3. To enhance system administration skills by working with commands across different operating systems.

❖ **Requirements:**

Hardware Requirements:

- **Processor:** Multi-core CPU, Intel Core i3 (3.0 GHz) or higher
- **RAM:** Minimum 4 GB (8 GB recommended for optimal performance)
- **Storage:** 100 GB HDD or SSD (Solid State Drive) for faster access
- **Network Interface:** Ethernet or Wi-Fi adapter for connectivity



Software Requirements:

- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04/CentOS 8), UNIX-based OS
- **Command-line Interface:** PowerShell or Command Prompt (Windows), Terminal (Linux/UNIX)
- **Text Editor:** Nano, Vim, or Visual Studio Code for file editing
- **Administrative Privileges:** Superuser (Linux/UNIX) or Administrator (Windows) access

❖ **Theory:**

Command-line interfaces (CLI) are fundamental tools used in system administration to interact with operating systems efficiently. They enable users to execute tasks by entering text-based commands rather than relying on graphical interfaces. Using CLI commands, administrators can manage files and directories, modify permissions, monitor system performance, and configure system resources. Operating systems such as Windows, Linux, and UNIX offer numerous built-in commands to simplify administrative operations. Commands like `ls`, `cd`, `pwd`, `mkdir`, and `chmod` assist in navigating and organizing the file system. Understanding command structure and syntax ensures accurate execution of tasks. CLI usage helps in performing tasks faster and with greater control. It also supports automation through scripts, improving productivity and consistency. Gaining proficiency in basic commands enhances system management, security, and troubleshooting skills. This practical aims to build a strong foundation in using command-line tools effectively.

❖ **Commands:**

1. Display User Manual of a Command

- Functionality: Shows the manual page with details about a command's usage, options, and arguments.

- Syntax: `man <command>`

- Example: `man ls`

2. Change Current Working Directory.

- Functionality: Changes the terminal's current working directory.

- Syntax: `cd <directory-path>`

- Example: `cd /home/user/Documents.`

3. List Contents of the Current Directory.

- Functionality: Lists all files and directories in the current location.

- Syntax: `ls`

- Example: `ls`

4. Read/Modify/Concatenate Text Files.

- Functionality: Displays or manipulates file content.

- Syntax:

- Read: `cat <filename>`

- Modify: `'nano <filename>`

- Concatenate: `cat <file1> <file2> > <outputfile>`

5. Create a New Directory.

- Functionality: Creates a new directory at the specified path.

- Syntax: `mkdir <directory-name>`

- Example: `mkdir newdir`

6. Display Current Working Directory.

- Functionality: Prints the current directory path.

- Syntax: `pwd`

- Example: `pwd`

7. Write Arguments to Standard Output.

- Functionality: Prints the provided string or variables.

- Syntax: `echo <arguments>`

-Example: `echo Hello World`

8. Remove a File.

- Functionality: Deletes a specified file.
- Syntax: `rm <filename>`
- Example: `rm file.txt`

9. Delete a Directory.

- Functionality: Removes an empty directory.
- Syntax: `rmdir <directory-name>`
- Example: `rmdir olddir`

10. Copy a File or Directory.

- Functionality: Copies a file or directory to a destination.
- Syntax: `cp <source> <destination>`
- Example: `cp file.txt backup/`

11. Switch to Root User.

- Functionality: Gains root privileges temporarily.
- Syntax: `sudo su`
- Example: `sudo s`

12. Move Files or Directories.

- Functionality: Moves or renames files and directories.
- Syntax: `mv <source> <destination>`
- Example: `mv file.txt newdir/`

13. Search for a String in a File.

- Functionality: Searches for a specific word or pattern in a file.
- Syntax: `grep "<string>" <file>`
- Example: `grep "error" log.txt`

14. Print Top N Lines of a File.

- Functionality: Displays the first N lines of a file.
- Syntax: `head -n <N> <file>`
- Example: `'head -n 10 file.txt'`

15. Print Last N Lines of a File.

- Functionality: Displays the last N lines of a file.
- Syntax: `tail -n <N> <file>`
- Example: `'tail -n 10 file.txt'`

16. Remove Read Permission from Owner.

- Functionality: Revokes the owner's read permission for a file.
- Syntax: `chmod u-r <filename>`
- Example: `chmod u-r file.txt`

17. Change Specific Permissions.

- Functionality: Sets or removes specific file permissions.
- Syntax: `chmod u+r,w-x,g+w <filename>`
- Example: `chmod u+r,w-x,g+w file.txt`

18. Add Write Permission to Owner, None to Others.

- Functionality: Allows write access for the owner only.
- Syntax: `chmod u+w,o-rwx <filename>`
- Example: `chmod u+w,o-rwx file.txt`

19. Assign Permissions to Users.

- Functionality: Modifies file access for users, groups, and others.
- Syntax: `chmod u+rwx,g+rx,o+r <filename>`
- Example: `'chmod u+rwx,g+rx,o+r file.txt`

20. Assign R/W/X to Others.

- Functionality: Gives read, write, and execute permissions to others.
- Syntax: `chmod o+rwx <filename>`
- Example: `chmod o+rwx file.txt`

21. Remove All Permissions from All Users.

- Functionality: Clears all permissions on a file.
- Syntax: `'chmod a-rwx <filename>`
- Example: `'chmod a-rwx file.txt`

22. Remove Read Permission Using Absolute Mode.

- Functionality: Uses numeric mode to restrict read access.
- Syntax: `chmod 700 <filename>`
- Example: `chmod 700 file.txt`

23. Set R/W for Owner, None for Group/Other.

- Functionality: Assigns permissions in numeric mode.
- Syntax: `chmod 600 <filename>`
- Example: `chmod 600 file.txt'`

24. Add Execute for Owner, Read for Group/Others.

- Functionality: Adds execution and read access.
- Syntax: `chmod u+x,g+r,o+r <filename>`

- Example: `chmod u+x,g+r,o+r file.txt`

25. Add Execute Permission to All Users.

- Functionality: Enables execution by everyone.
- Syntax: `chmod a+x <filename>`
- Example: `chmod a+x script.sh`

OUTPUT:-

```
students@M307-59:~$ nano os_lab_practical.sh
students@M307-59:~$ chmod +x os_lab_practical.sh
./os_lab_practical.sh
===== Operating System Lab Practical =====
1. man ls (Manual page)
LS(1)                                User Commands                                LS(1)

NAME
ls - list directory contents

2. Change Directory
3. List Files
Desktop Downloads os_lab_practical.sh Public Templates
Documents Music Pictures snap Videos
4. File Operations
Hello World
5. Create Directory
6. Print Working Directory
/home/students
7. Echo Command
Hello Ubuntu
8. Remove File
9. Remove Directory
10. Copy File
11. sudo su (Run manually if needed)
12. Move File
13. Grep Search
Hello World
14. Head Command
Hello World
Operating System Lab
```

```
6. Print Working Directory
/home/students
7. Echo Command
Hello Ubuntu
8. Remove File
9. Remove Directory
10. Copy File
11. sudo su (Run manually if needed)
12. Move File
13. Grep Search
Hello World
14. Head Command
Hello World
Operating System Lab
15. Tail Command
Hello World
Operating System Lab
16. chmod u-r
17. chmod specific
chmod: invalid mode: 'u+r,w-x,g+w'
Try 'chmod --help' for more information.
18. chmod owner write only
19. chmod user/group/others
20. chmod others rwx
21. chmod remove all
22. chmod 700
23. chmod 600
24. chmod execute owner
25. chmod a+x
===== Practical Completed Successfully =====
students@M307-59:~$
```

Operating System Lab (N-PCCCM401P)

```
Feb 16 5:15 PM
students@M307-59: ~
GNU nano 7.2 os_lab_practical.sh
#!/bin/bash

echo "===== Operating System Lab Practical ====="

# 1. Display User Manual of a Command
echo "1. man ls (Manual page)"
man ls | head -n 5

# 2. Change Current Working Directory
echo "2. Change Directory"
cd ~

# 3. List Contents of the Current Directory
echo "3. List Files"
ls

# 4. Read/Modify/Concatenate Text Files
echo "4. File Operations"
echo "Hello World" > file1.txt
echo "Operating System Lab" > file2.txt
cat file1.txt
cat file1.txt file2.txt > combined.txt

# 5. Create a New Directory
echo "5. Create Directory"
mkdir newdir

[ Read 109 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

```
Feb 16 5:15 PM
students@M307-59: ~
GNU nano 7.2 os_lab_practical.sh

# 6. Display Current Working Directory
echo "6. Print Working Directory"
pwd

# 7. Write Arguments to Standard Output
echo "7. Echo Command"
echo Hello Ubuntu

# 8. Remove a File
echo "8. Remove File"
rm file2.txt

# 9. Delete a Directory
echo "9. Remove Directory"
mkdir olddir
rmdir olddir

# 10. Copy a File or Directory
echo "10. Copy File"
cp file1.txt newdir/

# 11. Switch to Root User (Commented for safety)
echo "11. sudo su (Run manually if needed)"
# sudo su

# 12. Move Files or Directories
```

```
Feb 16 5:16 PM
students@M307-59: ~
GNU nano 7.2 os_lab_practical.sh

# 20. Assign R/W/X to Others
echo "20. chmod others rwx"
chmod o+rwx newdir/file1.txt

# 21. Remove All Permissions from All Users
echo "21. chmod remove all"
chmod a-rwx newdir/file1.txt

# 22. Remove Read Permission Using Absolute Mode
echo "22. chmod 700"
chmod 700 newdir/file1.txt

# 23. Set R/W for Owner, None for Group/Other
echo "23. chmod 600"
chmod 600 newdir/file1.txt

# 24. Add Execute for Owner, Read for Group/Others
echo "24. chmod execute owner"
chmod u+x,g+r,o+r newdir/file1.txt

# 25. Add Execute Permission to All Users
echo "25. chmod a+x"
chmod a+x newdir/file1.txt

echo "===== Practical Completed Successfully ====="
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

❖ **Conclusion:** In conclusion, understanding and using essential operating system commands like **'ls', 'cd', 'cp', 'mv', and 'chmod'** enables efficient file management, navigation, and permission control. Tools like **'grep', 'head', and 'tail'** enhance data processing. Mastery of these commands improves system administration, task automation, and overall system security and performance.

❖ **Discussion Questions:**

1. **What is the significance of the pwd command in a Linux environment?**
2. **-Explain the function of the cp command and its common options.**
3. **How does chmod 700 affect file permissions, and what does each digit represent?**
4. **Describe the difference between head and tail commands in Linux.**
5. **What is the purpose of the grep command, and how is it used with regular expressions?**

❖ **References:**

<https://ubuntu.com/tutorials/command-line-for-beginners#1-overview>
<https://www.geeksforgeeks.org/25-basic-ubuntu-commands/>

Date:

Signature

Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26